QFF 2026 Phase 2 — PS-O-OI Bio-Sniff: Team Objectives and Deliverables

1. Purpose

Team Bio-Sniff continues its own Open Innovation problem (PS-O-OI) in Phase 2, and is judged on the same five objectives and the same 100-point rubric as the PS-O1, PS-O2 and PS-O3 teams. The problem itself is not refined. This document only fixes what the team must deliver, so that judges can score Bio-Sniff side by side with the other Optimization teams.

Read it together with the **QFF 2026 Optimization Challenge Kit** and its starter pack (`qff_phase2.py`). Everything in the kit applies to this team except Sections 6–8 (the O1/O2/O3 instances), which this document replaces. Where the two differ, this document governs for PS-O-OI.

Because Bio-Sniff selects exactly 3 of 9 candidates under a penalty, it has the same structure as PS-O2 (select exactly k assets). Its quality metrics and judging therefore follow the PS-O2 definitions.

2. The team's Phase 1 problem (as submitted)

Bio-Sniff uses QAOA to choose 3 of 9 candidate sensor positions in a paddy field for early detection of crop-disease volatile compounds. The data and placement scores are synthetic; the team states no field validation and no quantum-advantage claim. Summary from the team's repository:

Item	Phase 1 value
Variables	$x_i \in \{0,1\}$, $i = 1\ldots9$ ($x_i = 1$: candidate i selected); 9 qubits
Constraint	exactly 3 selected ($\Sigma x_i = 3$); 84 valid placements
Objective (maximise)	$F(x) = 0.60\,U(x) + 0.25\,C(x) + 0.15\,S(x)$: utility, pairwise complementarity, spatial spread
QUBO (minimise)	$E(x) = -F(x) + P\,(\Sigma x_i - 3)^2$, penalty $P = 3.0$
Algorithm	QAOA p = 1, COBYLA (max 60 iterations), 5 seeds per problem size

Item	Phase 1 value
Settings	4096 shots, seed 42, opt. level 1, basis rz/sx/x/cx, official noise (0.001 / 0.005 / 0.01)
Processors	A (5 qubits) for a 5-candidate version; B (9 qubits) for the full 9-candidate problem
Phase 1 noisy results	A, 5 cand.: valid 0.604, P_opt 0.065 · B, 5 cand.: valid 0.606, P_opt 0.067 · B, 9 cand.: valid 0.438, P_opt 0.0042, best [1, 6, 8]

The Phase 1 settings and Processor A/B maps already match the official ones in the kit, so no re-work is needed there. What Phase 1 did not have: a Bell-state test, Processor D, a random baseline and a penalty sweep. Phase 2 adds these.

3. What is fixed and what the team locks

The team keeps its own problem, but it must freeze the instance in the first 2 hours so judges compare a fixed problem.

Item	Rule for Phase 2
Official settings	Exactly as in Kit Section 3: 4096 shots, seeds 42, opt. level 1, SABRE, basis rz/sx/x/cx, official noise model. Use `qff_phase2.py` unchanged
Processors A and B	Official maps (Kit Section 3). Bio-Sniff's Phase 1 maps are identical
Processor D	Team design under Kit Section 5 rules
Main instance L	The 9-candidate problem from Phase 1: the same U, C, S scores, weights 0.60 / 0.25 / 0.15, k = 3, reference penalty P = 3.0
Small instance S	The 5-candidate problem from Phase 1 (same scores restricted to those 5 candidates), k = 3
Lock	Commit `data/instance.json` within the first 2 hours of the hackathon and do not change it afterwards (Kit Section 2 rule for Open Innovation)

`data/instance.json` must contain, for both L and S: the candidate list (index → qubit), every U, C and S value used (or the exact formula and inputs that produce them), the weights, k, P, and the exact classical optimum F* with its candidate set. Instance S must

list which 5 of the 9 candidates it uses.

Why two instances: instance L needs 9 qubits and does not fit Processor A (5 qubits). Under the kit's capacity rule, Processor A is reported as "not executable: needs 9 qubits, has 5" for L. Instance S was already a locked part of the team's Phase 1 work, so it is kept as the one problem that runs on all three processors. It is not a shrunken version made to force a fit.

Changing the weights, adding candidates or using a different score set is allowed only as a clearly labelled additional experiment.

4. Objective 1 — Bell-state preparation and verification

Identical to the other Optimization teams (Kit Section 4). Run `bell_experiment()` from `qff_phase2.py` on your Processor D:

1. One adjacent pair (distance 1) and the pair farthest apart in your D graph.
2. Each pair ideal and noisy, measured in ZZ, XX and YY, 4096 shots per basis (12 circuits).
3. Report ⟨XX⟩, ⟨YY⟩, ⟨ZZ⟩ and F, each ± SE, and the CX count after transpilation, in `results/tables/bell.csv`.
4. Explain in 3–5 sentences why noisy F drops and why the distant pair is worse.

```
F = \frac{1 + \langle XX\rangle - \langle YY\rangle + \langle ZZ\rangle}{4}
```

Check: with the template Processor D, the adjacent pair gives F = 1.000 ideal and about 0.963 noisy.

5. Objective 2 — Processor D design

Same rules as the other teams (Kit Section 5). Bio-Sniff's required qubit count is **9**, the same as PS-O3, so `validate_processor_d(D, required_qubits=9)` must return no violations.

Rule	Limit
Qubits	9 to 12
Connectivity	Connected, undirected graph of your choice
Maximum degree	4 links per qubit
Gate set	rz, sx, x, cx

Rule	Limit
Noise	Official model; any rates you list may be equal or worse, never better
Freeze	`processors/processor_D.json` at the final commit produces every D result

The written justification has the same four parts as for every team: a drawing of D; the bottleneck you measured on B for instance L (SWAPs, CX overhead, depth); how D addresses it, checked against your D results; one trade-off D creates.

A useful starting point: in Bio-Sniff's QUBO, the penalty $(\Sigma x_i - 3)^2$ alone couples every pair of candidates, so the cost layer needs all 36 pairs. Explain how your D topology handles that all-to-all interaction within the degree-4 limit.

6. Objective 3 — Solution and benchmarking

The team solves its locked instances with QAOA and benchmarks them on Processors A, B and D, ideal and noisy, against the exact classical optimum and a uniform-random baseline. All definitions below are the PS-O2 definitions, with "portfolio" read as "sensor placement".

Required runs

Run	Instance	Processors	Conditions	Rows
Main benchmark	L (9 candidates, P = 3)	A ("not executable: needs 9 qubits, has 5"), B, D	ideal, noisy	4 + A row
All-processor comparison	S (5 candidates, P = 3)	A, B, D	ideal, noisy	6
Penalty sweep	L	D	ideal, noisy	14

- QAOA depth p = 1 (p = 2 optional, as a separate row).
- At least 5 initializations with numpy `default_rng(seed)`, seeds 0–4, γ and β drawn uniformly from $[0, \pi]$. Any optimizer; report its name, iteration limit, and whether the cost used a statevector or sampled shots. Phase 1 used COBYLA with 60 iterations; one 9-candidate seed hit that limit, so raising the limit is recommended.

- Optimize the angles once in ideal simulation, then evaluate the same angles on every processor, ideal and noisy, 4096 shots each. Re-optimizing per processor is an allowed additional experiment.
- Penalty sweep: P = 0, 0.5, 1, 2, 3, 5, 10, re-optimizing the angles for each P. For each P report whether the QUBO minimum is a valid placement, valid frequency, P_opt and objective ratio, and explain the feasibility-versus-quality trade-off.

Definitions you must use

- **Valid placement**: a bitstring with exactly 3 candidates selected.
- **Exact classical optimum F***: the largest F(x) over all valid placements (84 for L, 10 for S), by enumeration. It is not the QUBO minimum, which can be invalid when P is small.
- **Selected placement**: the most frequently sampled valid placement. Also report the best F among sampled valid placements.
- **Objective ratio** = F(selected placement) / F*. The team's Phase 1 gap is 100 × (1 − objective ratio) and may be reported as well.
- **Valid frequency** = shots giving a valid placement / 4096.
- **P_opt** = shots giving the optimal placement / 4096.
- **Consistency** = mean and standard deviation of the objective ratio over the 5+ initializations.

```
F(x) = 0.60\,U(x) + 0.25\,C(x) + 0.15\,S(x), \qquad \min_x \; -F(x) + P\Big(\sum_i x_i - 3\Big)^{2}
```

If any U, C or S value can be negative, say so in `instance.json`; the objective ratio is used only when F* > 0.

Random baseline (required)

QAOA is useful only if it beats uniform random sampling of all bitstrings. Report this baseline beside every quality metric:

Metric	Instance L (2^9 = 512 bitstrings)	Instance S (2^5 = 32 bitstrings)
Valid frequency	84 / 512 = 0.164	10 / 32 = 0.313
P_opt (one optimal placement)	1 / 512 = 0.0020	1 / 32 = 0.031

Metric	Instance L (2^9 = 512 bitstrings)	Instance S (2^5 = 32 bitstrings)
Objective ratio	mean F over the 84 valid placements / F* (compute from your data)	mean F over the 10 valid placements / F*

For comparison, the Phase 1 noisy Processor B run of instance L reached valid frequency 0.438 and P_opt 0.0042, about 2.7× and 2.1× the random values.

Resource metrics

Use `circuit_metrics()` and the definitions in Kit Section 9: logical CX, CX count, depth, two-qubit depth, SWAP count and CX overhead, for every processor row of instances L and S.

7. Objectives 4 and 5 — Reproducibility, analysis and report

The reproducibility rules are those of Kit Section 10, unchanged:

1. A judge runs `pip install -r requirements.txt`, then `main.ipynb` top to bottom with no edits.
2. The notebook finishes within 30 minutes on a laptop CPU; slow steps (such as the penalty sweep) save their output in `results/` behind a `RECOMPUTE = False` flag.
3. Every seed is written in the notebook; required runs use the official seeds.
4. Numbers in the report match the notebook output.
5. AI tool use is disclosed in the README.

The final report (4–8 pages, `report.md` or `report.pdf`) has the same five parts as every team: problem and formulation; Bell results; Processor D design and justification; results on A, B and D against F* and the random baseline; trade-offs, limitations and next improvements.

For Bio-Sniff the analysis must also answer three questions in plain words:

- Did QAOA place sensors better than random placement, on which processors, and by how much?
- Which penalty P gives the best balance of valid placements and placement quality, and what happens at P = 0?
- What does the synthetic data limit? State what would be needed before the placement could be trusted in a real field.

8. Results table and repository layout

Use the same `results/tables/results.csv` columns as every Optimization team (Kit Section 9), so judges read all teams' tables the same way. One row per instance × processor × condition (× P for the sweep). Put the instance in `problem_id` (`PS-O-OI-L`, `PS-O-OI-S`) and the penalty in an extra last column `penalty`.

```
problem_id,processor,condition,executable,logical_qubits,physical_qubits,logi
cal_cx,cx,depth,two_q_depth,swaps,cx_overhead,shots,quality_metric,quality_va
lue,p_opt,classical_optimum,random_baseline,penalty
PS-O-OI-L,B,noisy,true,9,9,<n>,<n>,<n>,<n>,<n>,<n>,4096,objective_ratio,
<value>,<value>,<F*>,<random ratio>,3
PS-O-OI-L,B,noisy,true,9,9,<n>,<n>,<n>,<n>,<n>,<n>,4096,valid_frequency,
<value>,<value>,<F*>,0.164,3
PS-O-OI-L,A,ideal,false,9,5,,,,,,,,,,,,3
```

The sample rows show only the format. Each processor × condition has two quality rows: `objective_ratio` and `valid_frequency`.

```
project/
├── README.md               # team, PS-O-OI, how to run, key results table
├── main.ipynb              # runs everything top to bottom
├── requirements.txt        # pinned versions actually used
├── qff_phase2.py           # organizer starter module (unchanged)
├── data/instance.json      # locked instances L and S (first 2 hours)
├── processors/
│   ├── processor_A.json
│   ├── processor_B.json
│   └── processor_D.json    # frozen design
├── results/
│   ├── tables/results.csv
│   ├── tables/bell.csv
│   ├── tables/penalty_sweep.csv
│   └── figures/
└── report.md               # or report.pdf
```

The existing Phase 1 folders (`notebooks/`, `docs/`, dataset audit files) can stay in the repository. Judges run only `main.ipynb`.

9. How Bio-Sniff is judged

Bio-Sniff is scored on the Optimization rubric and score sheet used for every Optimization team. Wherever the rubric has a PS-O2 line, judges read it for PS-O-OI.

Objective	Points	What judges check for Bio-Sniff
1. Bell state	10	Two pairs on D, ideal and noisy, XX/YY/ZZ, F ± SE, interpretation (Section 4)
2. Processor D	20	Passes `validate_processor_d(D, 9)`; targets the B bottleneck measured on instance L; trade-off stated (Section 5)
3. Solution and benchmarking	35	Formulation 10: QUBO with the locked weights and P, F* by enumeration, instance locked in time. Quality 15: objective ratio, valid frequency and P_opt against the random baseline; penalty sweep. Resources 10: CX, depth, SWAPs on A/B/D (Section 6)
4. Validation and reproducibility	15	`main.ipynb` runs end to end; files present; numbers match the report (Section 7)
5. Analysis and report	20	The three Bio-Sniff questions answered with evidence; limitations of synthetic data (Section 7)

Objective points (6 of the 15 quality points). Taken from the instance L, Processor B, noisy row, the same row used for PS-O2. The reference for valid frequency is the team's own Phase 1 result (0.438), so matching the Phase 1 valid frequency earns the full 3 points for it, and random sampling earns 0.

```
\text{points} = 3\,\mathrm{clip}\!\left(\frac{v - 0.164}{0.438 - 0.164}, 0, 1\right) + 3\,\mathrm{clip}\!\left(\frac{r - r_{\min}}{1 - r_{\min}}, 0, 1\right)
```

Here v is the valid frequency, r the objective ratio, and r_min = F(worst valid placement) / F*, computed by the judges from `data/instance.json`.

Tie-breaks and automatic deductions are those of Kit Section 11: −2 for each missing required run (up to −10), Objective 3 capped at half if the locked instance or an official setting is changed in a required run, and Objective 2 = 0 if Processor D fails validation. Changing `data/instance.json` after the first 2 hours counts as changing the locked instance.

10. Final checklist before 5:40 PM on Day 2

- [ ] Same team and same problem (Bio-Sniff, PS-O-OI) as Phase 1, named in README.md

- [ ] `data/instance.json` with instances L and S committed in the first 2 hours and unchanged since
- [ ] Official settings used for every required run; `qff_phase2.py` unchanged
- [ ] Bell state: 2 pairs × ideal/noisy × XX, YY, ZZ with F ± SE in `results/tables/bell.csv`
- [ ] `processors/processor_D.json` passes `validate_processor_d(D, 9)` and is frozen; drawing and justification in the report
- [ ] Instance L on B and D (A marked "not executable"), instance S on A, B and D, all ideal and noisy, in `results/tables/results.csv`
- [ ] Penalty sweep P = 0, 0.5, 1, 2, 3, 5, 10 on D, ideal and noisy, in `results/tables/penalty_sweep.csv`
- [ ] F* by enumeration and the random baseline reported beside every quality metric
- [ ] `main.ipynb` runs top to bottom from a clean install within 30 minutes
- [ ] Report answers the three Bio-Sniff questions and states the synthetic-data limits
- [ ] AI use disclosed in README.md, if any
- [ ] Repository public and its URL submitted on the portal

Questions during the event: Optimization help desk at the venue, or qiskitfallfest@srmap.edu.in.